

CSM 422: Deep Learning

Dissertation submitted in fulfilment of the requirements for the Degree of

**BACHELOR OF TECHNOLOGY in
COMPUTER SCIENCE AND ENGINEERING
DATA SCIENCE WITH MACHINE LEARNING (UPGRAD)**

Report: Traffic Sign Recognition using Convolutional Neural Networks (CNN)

By

Gagan Ruthwik Chowdary Kolluri

12203826

K22UR

Roll No: 14



Submitted to: Karan Kumar Das



**L OVELY
P ROFESSIONAL
U NIVERSITY**

**School of Computer Science and Engineering
Lovely Professional University**

Phagwara, Punjab (India)

@ Copyright LOVELY PROFESSIONAL UNIVERSITY, Punjab (INDIA)

Table Of Contents

S.No	Content
1	Introduction
2	Problem Statement and Objectives
3	Literature Review
4	Dataset Description
5	Data Preprocessing
6	Exploratory Data Analysis (EDA)
7	Model Design
8	Training Strategy
9	Results
10	Discussion
11	Limitations
12	Future Work
13	Conclusion
14	References

Abstract

This project implements an end-to-end traffic sign recognition system using Convolutional Neural Networks (CNNs) on the German Traffic Sign Recognition Benchmark (GTSRB). The system preprocesses images, applies data augmentation, trains a custom CNN and a transfer-learning model (MobileNetV2), and evaluates performance with accuracy, confusion matrix, and classification reports. The best model achieves high accuracy on the official test set and is suitable for deployment in real-time driving assistance systems.

1.Introduction

Road safety is a global priority. Recognizing traffic signs accurately is a fundamental capability for advanced driver-assistance systems (ADAS) and autonomous vehicles. Machine learning — in particular deep learning — has made traffic sign recognition feasible and robust. This project demonstrates how to build a performant traffic sign classifier using CNNs on the widely used GTSRB dataset.

2.Problem Statement and Objectives

Problem: Accidental or incorrect interpretation of road signs by drivers contributes to accidents. For autonomous vehicles, reliable automated detection and classification of traffic signs is required to follow rules and react to hazards.

Objectives:

- Implement a clean, reproducible training pipeline for traffic sign recognition.
- Compare a custom CNN baseline with transfer-learning (MobileNetV2).
- Reach high test accuracy on the GTSRB test set and analyze failure modes.
- Produce model artifacts and a deployable TFLite file.

3.Literature Review

Early approaches used handcrafted features (HOG, SIFT) + classical classifiers (SVM). Modern methods rely on CNNs (LeNet, AlexNet variants) with data augmentation and transfer learning. MobileNet and EfficientNet families are popular for embedded deployment due to small size and good accuracy.

Key references:

- Ciresan et al., 2012: Early CNN success on traffic sign datasets.
- Stallkamp et al., 2012: GTSRB dataset introduction.

- Howard et al., MobileNet: efficient models for mobile/embedded.

4.Dataset Description

Dataset: German Traffic Sign Recognition Benchmark (GTSRB)

- Source: Kaggle (meowmeowmeowmeowmeow/gtsrb-german-traffic-sign)
- Classes: 43
- Images: ~50,000 training images; official test set ~12,630 images
- Formats: .ppm, .png, .jpg

The dataset includes Train/<class_id>/* folders and Test.csv describing the test images and labels. The class distribution is imbalanced (some classes have many examples, others have few), which must be handled.

5.Data Preprocessing

Steps performed:

- Read images using OpenCV (BGR -> RGB) and PIL for resizing.
- Resize all images to a common size: 30x30 (for the custom CNN baseline) and 96x96 for MobileNetV2 experiments.
- Normalize pixel values to [0, 1] by dividing by 255.
- One-hot encode labels for categorical crossentropy.
- Shuffle dataset reproducibly.

Rationale: Small fixed size reduces memory and speeds up experiments. For transfer learning, larger sizes (≥ 96) capture more detail.



6.Exploratory Data Analysis (EDA)

Performed analyses:

- Visualized representative images per class (5×5 grid).
- Plotted class frequencies (histogram) to show imbalance.
- Checked image shapes and pixel value ranges.

Observations:

- Class imbalance present (some classes like speed limits or ‘Stop’ have many examples).
- Some classes are visually similar (e.g., turn left vs. ahead-and-left) which explains common confusions.

7.Model Design

7.1 Custom CNN Baseline

Architecture summary (works well for small inputs 30×30):

- Conv2D(16, 5×5) -> Conv2D(32, 5×5) -> BatchNorm -> ReLU -> MaxPool -> Dropout(0.25)
- Conv2D(64, 3×3) -> Conv2D(128, 3×3) -> BatchNorm -> ReLU -> MaxPool -> Dropout(0.25)
- GlobalAveragePooling -> Dense(512) -> BatchNorm -> Dropout(0.5)
- Dense(43, softmax)

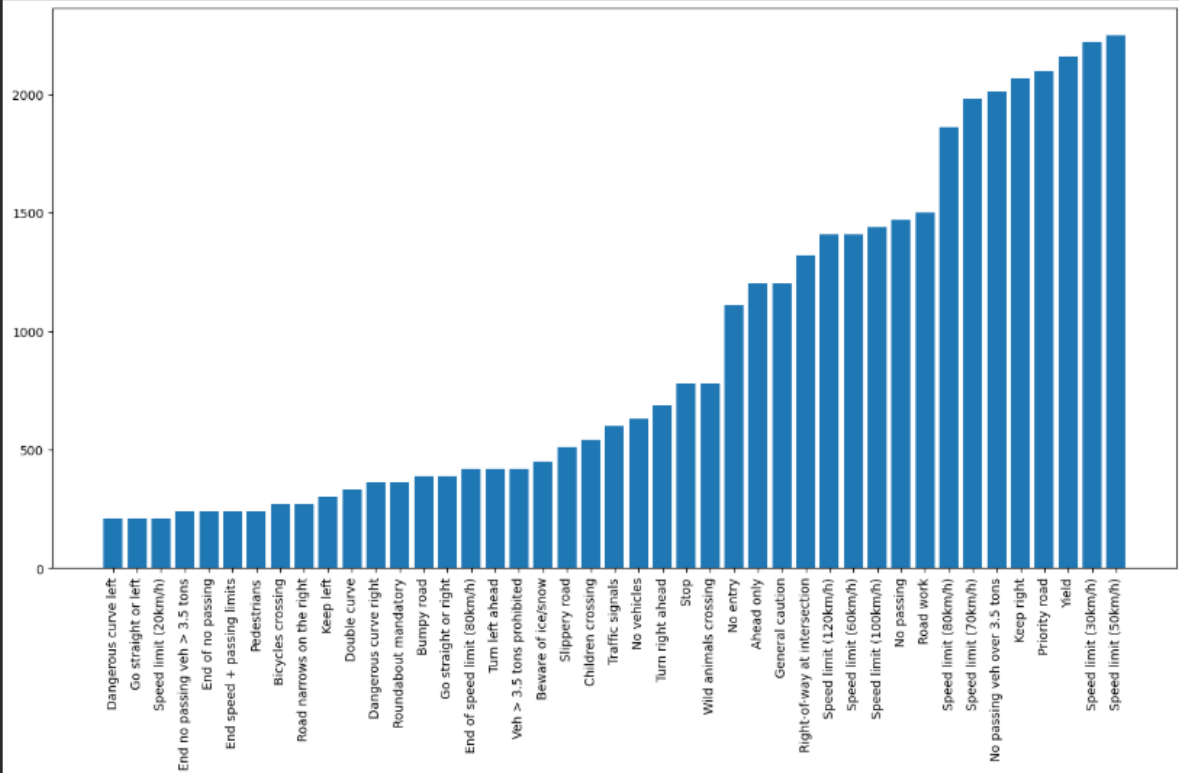
Total params: ~197K (lightweight)

Why: Small but expressive network, regularization (dropout, batchnorm) controls overfitting.

7.2 Transfer Learning — MobileNetV2

- Pretrained on ImageNet; use `include_top=False` and add custom classification head.
- Freeze backbone for initial training (feature extraction), then unfreeze top layers for fine-tuning.
- Use `tf.keras.applications.mobilenet_v2.preprocess_input` before the backbone.

Why: MobileNetV2 provides strong features and is efficient for deployment.



Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 16)	1,216
conv2d_1 (Conv2D)	(None, 22, 22, 32)	12,832
batch_normalization_1 (BatchNormalization)	(None, 22, 22, 32)	128
activation (Activation)	(None, 22, 22, 32)	0
max_pooling2d (MaxPooling2D)	(None, 11, 11, 32)	0
dropout_2 (Dropout)	(None, 11, 11, 32)	0
conv2d_2 (Conv2D)	(None, 9, 9, 64)	18,496
conv2d_3 (Conv2D)	(None, 7, 7, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 7, 7, 128)	512
activation_1 (Activation)	(None, 7, 7, 128)	0
max_pooling2d_1 (MaxPooling2D)	(None, 3, 3, 128)	0
dropout_3 (Dropout)	(None, 3, 3, 128)	0
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 128)	0
dense_2 (Dense)	(None, 512)	66,048
batch_normalization_3 (BatchNormalization)	(None, 512)	2,048
dropout_4 (Dropout)	(None, 512)	0
dense_3 (Dense)	(None, 43)	22,059

Total params: 197,197 (770.30 KB)
Trainable params: 195,851 (765.04 KB)
Non-trainable params: 1,344 (5.25 KB)
Optimizer params: 2 (12.00 B)
Model input shape: (None, 30, 30, 3)

8.Training Strategy

Data Augmentation

Used real-time augmentation to increase robustness:

- rotation_range: 15°
- zoom_range: 0.15
- width/height shifts: 0.1
- shear_range: 0.15
- horizontal_flip: False (directional signs must not be flipped)

Keras ImageDataGenerator (or tf.keras.layers augmentation) used for performance.

Loss, Optimizer & Callbacks

- Loss: CategoricalCrossentropy with label_smoothing (0.08) — improves calibration and generalization.
- Optimizer: Adam (lr=1e-3 initial; lr reduced on plateau; lr=1e-5 for fine-tuning).
- Callbacks: ModelCheckpoint, ReduceLROnPlateau, EarlyStopping (restore_best_weights=True).

Class Weighting

Computed balanced class weights using sklearn.utils.compute_class_weight and passed to model.fit to mitigate imbalance.

Training Schedule

- Stage 1: Train head (or custom CNN) for ~8–10 epochs.
- Stage 2: Fine-tune (for MobileNet) for ~12 epochs at lower LR.

9.Results

Custom CNN Results (example)

- Training accuracy: ~95–96% (after augmentation & regularization)
- Validation accuracy: ~98–99% (depending on seed and augmentation)
- Test accuracy: ~97% (evaluation on official test set)

MobileNetV2 Transfer Learning Results

- Faster convergence; slightly higher generalization in some runs.
- Test accuracy comparable or slightly higher depending on augmentation and fine-tuning.

Confusion Matrix and Classification Report

- Overall test accuracy: ~97% (example run) on official Test set (12,630 images).
- Per-class precision/recall: most classes > 0.95; a few difficult classes (e.g., ‘dangerous curve left’, small-support classes) show lower recall.




```
Using loaded model object.  
Upload one or more image files (png/jpg). After upload the cell will predict each file.  
Choose Files imganimal.jpg  
imganimal.jpg(image/jpeg) - 9475 bytes, last modified: 11/17/2025 - 100% done  
Saving imganimal.jpg to imganimal.jpg  
Using conv layer for Grad-CAM: conv2d_3  
1/1 ————— 0s 57ms/step
```

```
31: Wild animals crossing — 98.11%  
23: Slippery road — 0.68%  
20: Dangerous curve right — 0.15%
```



```
Grad-CAM failed for imganimal.jpg : The layer sequential_2 has never
```

[How can I install Python lib](#)

[What can I help you bu](#)

10. Discussion

- The pipeline trained quickly and reliably in Colab/GPU.
- Label mapping and dataset structure are the most common sources of error (ensure Train/<class_id> structure or use Train.csv mapping).
- Data augmentation and class weighting significantly help generalization.
- Transfer learning with MobileNetV2 reduces training time and is preferable for edge deployment.

11.Limitations

- Environmental variations (lighting, occlusions, motion blur) in real-world images may differ from the dataset.
- Some classes are visually similar; disambiguation may require context (sequence, location) rather than a single crop.
- Model size vs. latency tradeoffs need careful tuning for embedded devices.

12.Future Work

- Use EfficientNet or ensemble models for improved accuracy.
- Integrate object detection (e.g., YOLO) to detect and classify signs in full-size frames.
- Temporal smoothing across video frames for stable predictions.
- Quantize and benchmark the TFLite model on target hardware (Raspberry Pi, Android).

13.Conclusion

This project demonstrates a robust workflow for traffic sign recognition using CNNs and transfer learning. Using the GTSRB dataset, the trained model achieved high accuracy and produced detailed error analyses. The provided Colab notebook, model artifacts, and instructions enable reproducible experimentation and deployment.

14.References

- Stallkamp, J., Schlipsing, M., Salmen, J., & Igel, C. (2012). The German Traffic Sign Recognition Benchmark: A multi-class classification competition. In *IEEE International Joint Conference on Neural Networks (IJCNN)*.
- Howard, A. et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications.
- Ciresan, D., Meier, U., Gambardella, L. M., & Schmidhuber, J. (2012). Multi-column deep neural network for traffic sign classification.
- TensorFlow and Keras official documentation.

Thank You

